



IE3010

Network Programming

3rd Year, Semester II

Assignment Report

NetMessenger: A Multi-Client Chat and File-Sharing Platform over TCP/IP

Registration Number: IT23756632

Submitted to

Sri Lanka Institute of Information Technology

In partial fulfillment of the requirements for the
Bachelor of Science Special Honors Degree in Information Technology

07 October 2026

Declaration

I certify that this report does not incorporate without acknowledgement, any material previously submitted for a degree or diploma in any university, and to the best of my knowledge and belief it does not contain any material previously published or written by another person, except where due reference is made in text.

Registration Number : IT23756632

Name : Sithumini Gamage

Table of Contents

Declaration.....	i
Table of Contents.....	ii
List of Figures	iv
List of Tables	vii
1. Introduction.....	1
1.1 Scope of the Implementation	1
1.2 Development Environment	1
2. Personalisation and Assignment Compliance.....	2
2.1 Personalisation in Runtime Output	2
2.2 Source and Submission Structure	2
3. System Design	3
3.1 Server Concurrency Model	3
3.2 Shared State and Synchronisation.....	3
3.3 Client Design.....	4
4. Application Protocol	5
4.1 Framing Strategy	5
4.2 Error Responses	5
4.3 Disconnect Behaviour	6
5. Implementation Notes.....	7
5.1 Registration and Presence	7
5.2 Broadcast and Private Messaging	7
5.3 Room Management	7
5.4 File Transfer and Storage	7
5.5 Server Logging.....	7
5.6 Optional Rate Limiting	8
6. Verification and Test Evidence.....	9
6.1 Functional Test Summary	14
6.2 Requirement-to-Evidence Traceability	15
7. Optional Extension: Command Rate Limiting	16
7.1 Behavioural Rationale.....	16
8. Incremental Development and Version Control	17
8.1 Development Corrections	18

9. Design Decisions, Assumptions and Limitations	19
10. Verification Quality and Learning	20
11. Conclusion	21
Appendix A - Submission Package Evidence.....	22
Appendix B - Supporting Development Evidence.....	23
Appendix C - Annotated Source-Code Evidence	24

List of Figures

Figure 3.1: High-level architecture of NetMessenger

Figure 6.1: Final compilation and personalised server startup

Figure 6.2: Multi-client registration, LIST and listener verification

Figure 6.3: Duplicate username rejection and session continuity

Figure 6.4: Broadcast, private and room messaging

Figure 6.5: User-to-user file transfer and personalised storage

Figure 6.6: File integrity verification using cmp

Figure 6.7: Personalised storage path and timestamped server log

Figure 6.8: Invalid command handling

Figure 6.9: Rate-limiting behaviour under repeated commands

Figure 8.1: Feature commit pushed to GitHub

Figure 8.2: Incremental Git commit history

Figure A.1: Final submission ZIP content check

Figure B.1: Compiler warning identified and corrected before the final build

Figure C.1: Server accept loop and pthread-based client concurrency

Figure C.2: Line framing and exact-byte file reception

Figure C.3: Broadcast and private-message command handling

```
1 int room_message(int sender_index,
2                 const char *room_name,
3                 const char *message)
4 {
5     int target_sockets[MAX_CLIENTS];
6     int target_count = 0;
7
8     pthread_mutex_lock(&rooms_mutex);
9     int room_index = find_room_locked(room_name);
10
11     if (room_index < 0) {
12         pthread_mutex_unlock(&rooms_mutex);
13         return -1;
14     }
15
16     if (!rooms[room_index].members[sender_index]) {
17         pthread_mutex_unlock(&rooms_mutex);
18         return 0;
19     }
20
21     pthread_mutex_lock(&clients_mutex);
22
23     for (int i = 0; i < MAX_CLIENTS; i++) {
24         if (i != sender_index &&
25             rooms[room_index].members[i] &&
26             clients[i].registered) {
27             target_sockets[target_count++] =
28                 clients[i].socket_fd;
29         }
30     }
31
32     pthread_mutex_unlock(&clients_mutex);
33     pthread_mutex_unlock(&rooms_mutex);
34 }
```

Figure C.4: Room membership check and room-message routing

```

1  enterRoom(struct sockaddr_in *client_addr, 9) == 0) {
2      char target[MAX_USERNAME]; room_name,
3      char filename[MAX_FILENAME]; message)
4  {
5      unsigned long long file_size_value;
6      char text[300];
7      int target_count = 0;
8      int fields = sscanf(buffer,
9      pthread_mutex_lock(&rooms_mutex);
10     int target_index = find_index(room_name);
11
12     if (find_index < 0) {
13         pthread_mutex_unlock(&rooms_mutex);
14         return ERR105 INVALID_FORMAT %s\n", NID_TAG);
15     }
16     send_all(client_fd, response, strlen(response));
17     continue;
18     if (!rooms[room_index].members[sender_index]) {
19         pthread_mutex_unlock(&rooms_mutex);
20         if (file_size_value >
21             (unsigned long long)MAX_FILE_SIZE) {
22             pthread_mutex_lock(&clients_mutex);
23             (size_t)file_size_value) < 0) {
24                 for (int i = 0; i < MAX_CLIENTS; i++) {
25                     if (i != sender_index &&
26                         rooms[room_index].members[i] &&
27                         !strcmp(rooms[room_index].members[i], response),
28                         ERR004 ERR_TOO_MANY_MEMBERS NID_TAG);
29                     clients[i].socket_fd;
30                     send_all(client_fd, response, strlen(response));
31                 }
32                 continue;
33             }
34             pthread_mutex_unlock(&clients_mutex);
35             pthread_mutex_lock(&rooms_mutex);
36             file_size_value;
37         }
38     }
39 }

```

Figure C.5: SENDFILE parsing, size validation and framing protection

Figure C.6: Invalid-command response and common connection cleanup

List of Tables

Table 1.1: Development environment

Table 2.1: Personalised configuration values

Table 4.1: NetMessenger command set

Table 4.2: Main protocol error conditions

Table 6.1: Functional test summary

Table 6.2: Requirement-to-evidence traceability

Table 8.1: Incremental Git development history

1. Introduction

NetMessenger is a multi-client chat and file-sharing application implemented in C using the BSD sockets API. A single TCP server accepts several concurrent clients and provides registration, presence information, broadcast messaging, private messaging, chat rooms and file transfer. The program was built and tested on CentOS with GCC and POSIX threads. The implementation was developed feature by feature rather than as one large code drop. That approach made it possible to test protocol behaviour at each stage, correct compiler and runtime issues as they appeared, and preserve a meaningful Git history. Particular attention was given to details that are easy to miss in simple socket examples: persistent TCP sessions, command framing, exact-byte file reception, shared state between threads and cleanup after both normal and unexpected disconnections.

1.1 Scope of the Implementation

- One TCP server listening on the personalised port 12632.
- Multiple simultaneously connected clients handled using pthreads.
- Unique username registration and user presence management.
- Broadcast, private and room-based text communication.
- File transfer with personalised server-side storage.
- Timestamped server logging and protocol error responses.
- Optional per-client rate limiting as a simple flood-protection extension.

1.2 Development Environment

Table 1.1: Development environment

Item	Environment used
Operating system	CentOS Linux virtual machine
Language	C
Networking API	BSD sockets
Compiler	GCC
Concurrency	POSIX threads (pthread)
Version control	Git and GitHub
Test host	localhost / 127.0.0.1

2. Personalisation and Assignment Compliance

The assignment-specific values were calculated from registration number IT23756632 and used consistently throughout the program. For IT23756632, the last four digits are 6632; therefore the listening port is $6000 + 6632 = 12632$. The node identifier used in normal OK and ERR responses is NID:7566.

Table 2.1: Personalised configuration values

Configuration item	Value used
Registration number	IT23756632
Numeric part	23756632
Last four digits	6632
Server port	12632
Node identifier	NID:7566
Server source file	server_6632.c
Client source file	client_6632.c
Makefile	Makefile_6632
Log file	netmsg_IT23756632.log
Server storage	./storage/IT23756632/<sender_username>/<filename>
Final archive	IE3010_IT23756632.zip

2.1 Personalisation in Runtime Output

The personalised values are visible at runtime as well as in the filenames. The server prints the registration number at startup, listens on port 12632, appends NID:7566 to normal status and error replies, writes to the personalised log filename, and stores transferred files below the IT23756632 directory. These points are demonstrated in Section 6.

2.2 Source and Submission Structure

The project is organised around server_6632.c, client_6632.c and Makefile_6632. Supporting documentation consists of README.md, design_diary.md, prompt_log.md and reflection.md. The runtime log is retained as evidence of actual server activity. The implementation report is kept as a separate deliverable so it can be finalised without changing the tested source code. The GitHub repository used for version control is https://github.com/SithuminiGamage13/NetMessenger_IT23756632.

3. System Design

The design uses a central server because user discovery, room membership, logging and file routing all depend on shared state. Each client keeps one persistent TCP connection to the server. Once registered, the same connection is used for later commands until the client sends QUIT or the socket closes.

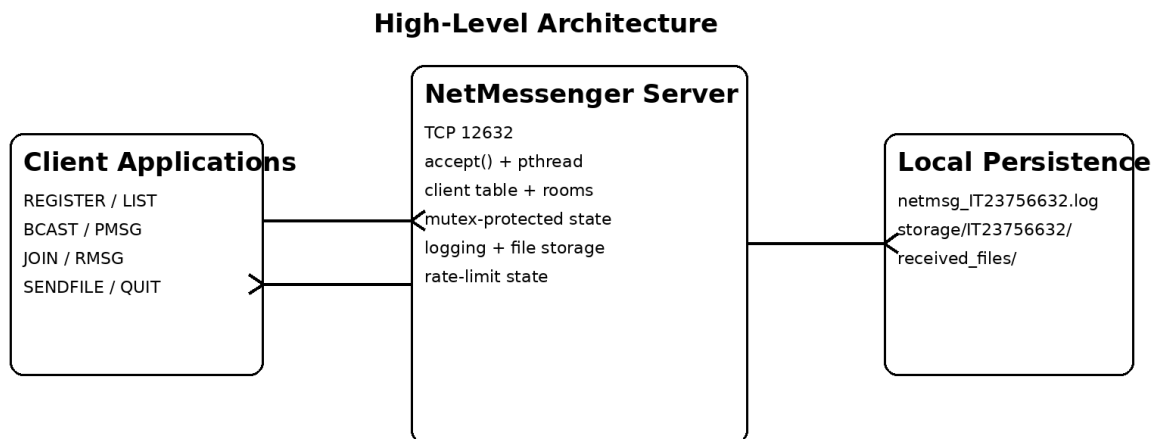


Figure 3.1: High-level architecture of NetMessenger

3.1 Server Concurrency Model

The main server thread creates the listening socket and blocks in accept(). Each accepted connection is assigned a client slot and processed by a separate pthread. This allows one client to wait for keyboard input while another is sending a message or file. A thread-per-client model was selected because it is direct, readable and appropriate for the scale of this assignment.

3.2 Shared State and Synchronisation

Client registrations and room memberships are shared between handler threads. Mutex protection is used when those structures are read or modified. Socket writes are also coordinated so that two threads do not write overlapping application messages to the same client connection.

3.3 Client Design

The client maintains a sending path for commands entered by the user and a receiving path for asynchronous data from the server. Incoming file data is handled separately from normal

line-based messages. This is necessary because a client may receive a private message, room message or file while waiting for the next keyboard command.

4. Application Protocol

Normal NetMessenger commands are newline-terminated text records. The server reads one complete command line, validates the keyword and arguments, performs the requested operation and returns an OK or ERR response where appropriate. File content is handled separately as raw bytes after the SENDFILE command.

Table 4.1: NetMessenger command set

Command	Purpose	Example
REGISTER <username>	Register a unique username	REGISTER sithumini
LIST	List connected registered users	LIST
BCAST <message>	Broadcast text to connected users	BCAST hello everyone
PMSG <user> <message>	Send a private message	PMSG alice hello privately
JOIN <room>	Create or join a room	JOIN general
LEAVE <room>	Leave a room	LEAVE general
ROOMS	List available rooms	ROOMS
RMSG <room> <message>	Send to room members	RMSG general hello room
SENDFILE <target> <file> <size>	Send file bytes to a target	SENDFILE alice testfile.txt 43
QUIT	Close the session gracefully	QUIT

4.1 Framing Strategy

TCP is a byte-stream protocol and does not preserve application message boundaries. The implementation therefore does not assume that one `recv()` call contains one complete command. Text commands are accumulated until a newline is received. When SENDFILE is accepted, an exact-byte receive loop is used until the declared file length has been consumed before command parsing resumes.

4.2 Error Responses

Table 4.2: Main protocol error conditions

Error	Condition
ERR 001 USERNAME_TAKEN	A second client attempts to register an existing username
ERR 002 USER_NOT_FOUND	A private-message target is not registered
ERR 003 ROOM_NOT_FOUND	A named room cannot be resolved
ERR 005 INVALID_COMMAND	The command keyword is not recognised
ERR 007 NOT_IN_ROOM	A user sends RMSG without current membership
ERR 010 RATE_LIMITED	The optional command-rate threshold is exceeded

4.3 Disconnect Behaviour

QUIT returns a final OK BYE response and then triggers cleanup. Unexpected socket closure follows the same cleanup path without the normal QUIT exchange. In both cases the client is removed from the active table and any room memberships are cleared.

5. Implementation Notes

5.1 Registration and Presence

A new connection begins unregistered. REGISTER first checks whether the requested username already belongs to another active client. Only a unique name is committed to the connected-client table. LIST builds its output from the currently registered entries. Presence messages inform other clients when a registered user joins or leaves.

5.2 Broadcast and Private Messaging

BCAST forwards the sender's message to the other registered clients. PMSG resolves one username and writes only to that user's socket. The sender receives a status response after the operation, while the receiving client is shown the sender and message type.

5.3 Room Management

JOIN adds the current client to a room, creating the room if required. ROOMS enumerates the available room table. LEAVE removes the user's membership without disconnecting the session. RMSG first checks both the room and the sender's membership before forwarding text to room members.

5.4 File Transfer and Storage

SENDFILE supplies a target, filename and byte count. The server reads exactly the declared number of bytes, creates the personalised storage directory when necessary, and writes the data below `./storage/IT23756632/<sender_username>/. The receiving client stores the forwarded copy below ./received_files/.`

The final test used a 43-byte testfile.txt. Correctness was verified with the Linux cmp utility against both the client-side copy and the server-side stored copy. An exit status of 0 was obtained in both comparisons.

5.5 Server Logging

The server writes timestamped operational events to netmsg_IT23756632.log. The recorded evidence includes server startup, successful registrations, file transfer, broadcast activity and graceful disconnects. The log provides a second source of evidence when several terminal windows are active.

5.6 Optional Rate Limiting

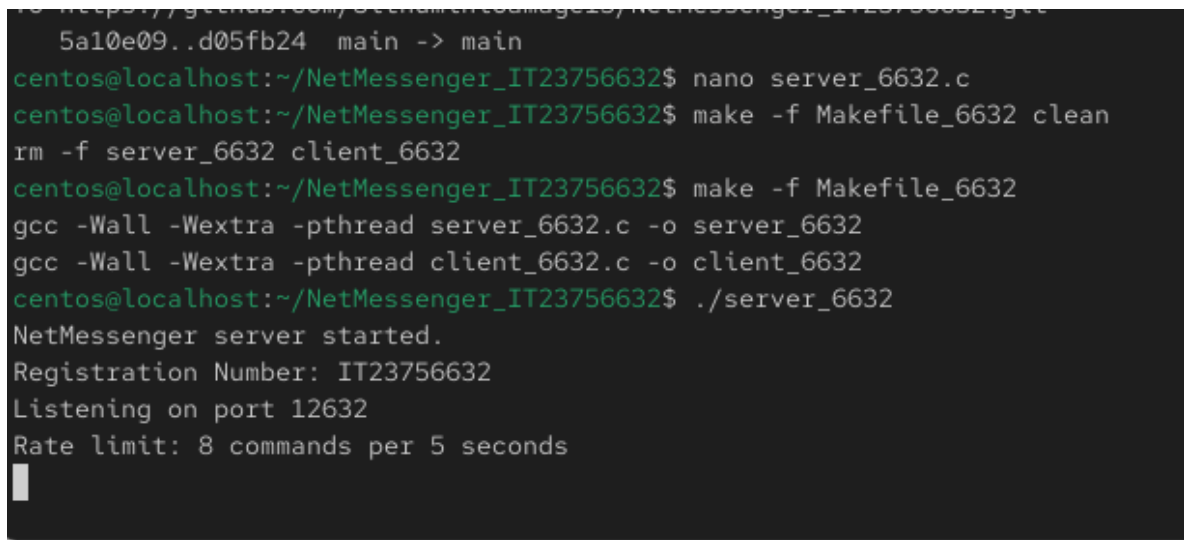
A simple per-client limiter allows up to eight text commands within five seconds. Additional commands during the same window are rejected with ERR 010 RATE_LIMITED NID:7566.

The state is maintained per client, so one user reaching the limit does not stop the server or disconnect other users.

Annotated source-code evidence for connection handling, concurrency, TCP framing, messaging, room routing, file transfer and error cleanup is included in Appendix C. These excerpts are taken from the final implementation and are paired with short explanations of the relevant logic.

6. Verification and Test Evidence

The following screenshots were captured during the actual CentOS development and test sessions. Each figure is paired with a short explanation of the behaviour it verifies.



```
5a10e09..d05fb24  main -> main
centos@localhost:~/NetMessenger_IT23756632$ nano server_6632.c
centos@localhost:~/NetMessenger_IT23756632$ make -f Makefile_6632 clean
rm -f server_6632 client_6632
centos@localhost:~/NetMessenger_IT23756632$ make -f Makefile_6632
gcc -Wall -Wextra -pthread server_6632.c -o server_6632
gcc -Wall -Wextra -pthread client_6632.c -o client_6632
centos@localhost:~/NetMessenger_IT23756632$ ./server_6632
NetMessenger server started.
Registration Number: IT23756632
Listening on port 12632
Rate limit: 8 commands per 5 seconds
```

Figure 6.1: Final compilation and personalised server startup

The clean build compiles `server_6632.c` and `client_6632.c` with `-Wall`, `-Wextra` and `-pthread`. The server then starts successfully, prints registration number `IT23756632`, listens on port `12632` and reports the configured rate limit.

```

Oct 7 10:38 AM

centos@localhost:~/NetMessenger_IT23756632 - ./server_6632
~/NetMessenger_IT23756632
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 3.60 KiB | 3.60 MiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/SithuminiGamage13/NetMessenger_IT23756632.git
d05fb24..78ce382 main -> main
centos@localhost:~/NetMessenger_IT23756632$ ./server_6632
NetMessenger server started.
Registration Number: IT23756632
Listening on port 12632
Rate limit: 8 commands per 5 seconds
Client connected from 127.0.0.1:60956 (slot 0)
Client 0 received: REGISTER sithumini
Client connected from 127.0.0.1:44172 (slot 1)
Client 1 received: REGISTER alice
Client 0 received: LIST
>
centos@localhost:~/NetMessenger_IT23756632 - ./client_6632
~/NetMessenger_IT23756632
OK REGISTERED alice NID:7566
>
File received from sithumini: ./received_files/sithumini_testfile.txt (43 bytes)
>
MSG BCAST SERVER sithumini left
> QUIT
>
OK BYE NID:7566
centos@localhost:~/NetMessenger_IT23756632$ ./client_6632
Connected to NetMessenger server 127.0.0.1:12632
> REGISTER alice
>
OK REGISTERED alice NID:7566
>

centos@localhost:~/NetMessenger_IT23756632
~/NetMessenger_IT23756632
> LIST
OK USERS sithumini NID:7566
>
> LIST
>
OK USERS sithumini NID:7566
> QUIT
OK BYE NID:7566
centos@localhost:~/NetMessenger_IT23756632$ ./client_6632
Connected to NetMessenger server 127.0.0.1:12632
> REGISTER sithumini
>
OK REGISTERED sithumini NID:7566
>
MSG BCAST SERVER alice joined
> LIST
>
OK USERS sithumini,alice NID:7566
>

centos@localhost:~/NetMessenger_IT23756632
~/NetMessenger_IT23756632
centos@localhost:~$ cd ~/NetMessenger_IT23756632
centos@localhost:~/NetMessenger_IT23756632$ make -f Makefile_6632 clean
rm -f server_6632 client_6632
centos@localhost:~/NetMessenger_IT23756632$ make -f Makefile_6632
gcc -Wall -Wextra -pthread server_6632.c -o server_6632
gcc -Wall -Wextra -pthread client_6632.c -o client_6632
centos@localhost:~/NetMessenger_IT23756632$ ss -tlnp | grep 12632
LISTEN 0      10          0.0.0.0:*        users:(("server_6632",pid=25669,fd=3))
centos@localhost:~/NetMessenger_IT23756632$

```

Figure 6.2: Multi-client registration, LIST and listener verification

Two clients are registered at the same time. LIST returns sithumini and alice with NID:7566, while ss confirms that the server process is listening on TCP port 12632. This verifies persistent sessions, concurrency and port personalisation.

```

Oct 7 5:30 AM

centos@localhost:~/NetMessenger_IT23756632 - ./server_6632
~/NetMessenger_IT23756632
Client 0 received: REGISTER sithumini
Registered user: sithumini
Client connected from 127.0.0.1:36840 (slot 1)
Client 1 received: REGISTER alice
Registered user: alice
Client 0 received: LIST
Client 1 received: LIST
Client connected from 127.0.0.1:57740 (slot 2)
Client 2 received: REGISTER sithumini
Client 0 received: QUIT
User sithumini disconnected gracefully.
Client 2 received: QUIT
Unregistered client disconnected.
Client 1 received: QUIT
User alice disconnected gracefully.
git status
centos@localhost:~/NetMessenger_IT23756632 - ./client_6632
~/NetMessenger_IT23756632
centos@localhost:~$ cd ~/NetMessenger_IT23756632
centos@localhost:~/NetMessenger_IT23756632$ ./client_6632
Connecting to NetMessenger server...
Connected successfully to 127.0.0.1:12632
> REGISTER alice
Server: OK REGISTERED alice NID:7566
> LIST
Server: OK USERS sithumini,alice NID:7566
> QUIT
Server: OK BYE NID:7566
centos@localhost:~/NetMessenger_IT23756632$

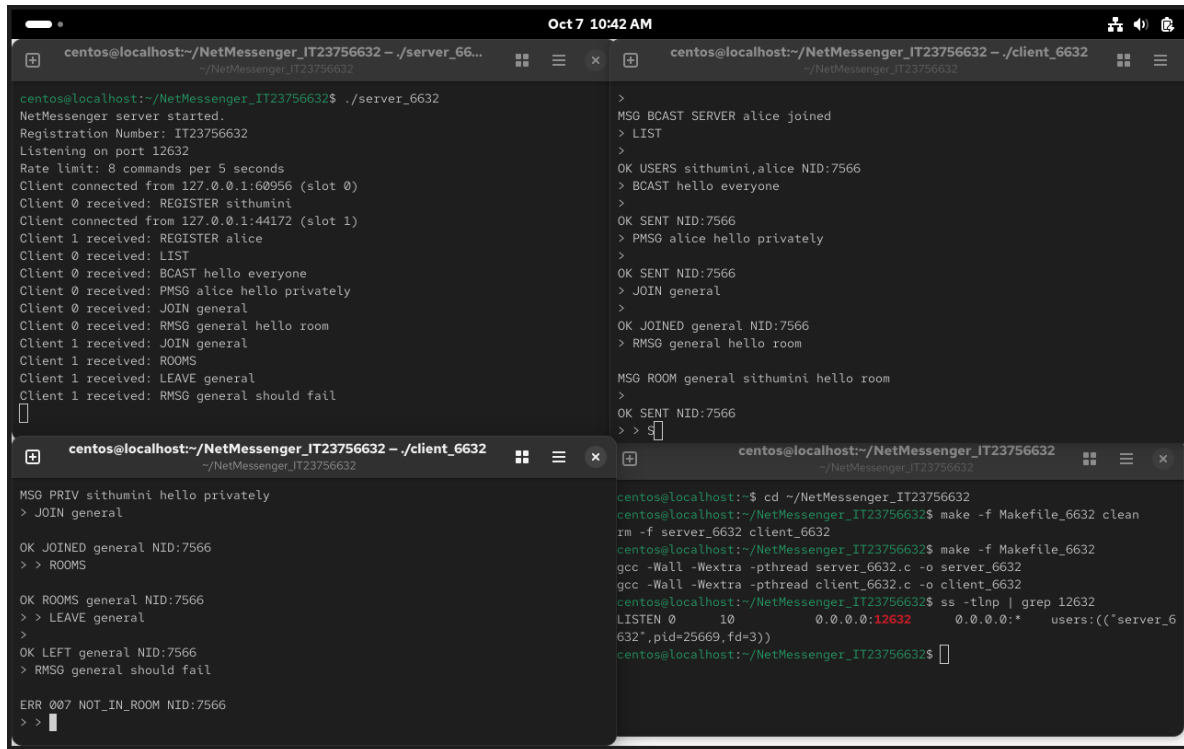
centos@localhost:~/NetMessenger_IT23756632
~/NetMessenger_IT23756632
centos@localhost:~$ cd ~/NetMessenger_IT23756632
centos@localhost:~/NetMessenger_IT23756632$ ./client_6632
Connecting to NetMessenger server...
Connected successfully to 127.0.0.1:12632
> REGISTER sithumini
Server: OK REGISTERED sithumini NID:7566
> LIST
Server: ERR 005 INVALID_COMMAND NID:7566
> QUIT
Server: OK BYE NID:7566
centos@localhost:~/NetMessenger_IT23756632$

centos@localhost:~/NetMessenger_IT23756632
~/NetMessenger_IT23756632
centos@localhost:~$ cd ~/NetMessenger_IT23756632
centos@localhost:~/NetMessenger_IT23756632$ ./client_6632
Connecting to NetMessenger server...
Connected successfully to 127.0.0.1:12632
> REGISTER sithumini
Server: OK REGISTERED sithumini NID:7566
> LIST
Server: ERR 001 USERNAME_TAKEN NID:7566
> QUIT
Server: ERR 005 REGISTER_REQUIRED NID:7566
centos@localhost:~/NetMessenger_IT23756632$

```

Figure 6.3: Duplicate username rejection and session continuity

A third client attempts REGISTER sithumini while that username is already active. The server returns ERR 001 USERNAME_TAKEN NID:7566. The existing clients remain connected and continue to use LIST normally.



```
centos@localhost:~/NetMessenger_IT23756632 - ./server_6632
~/NetMessenger_IT23756632
centos@localhost:~/NetMessenger_IT23756632$ ./server_6632
NetMessenger server started.
Registration Number: IT23756632
Listening on port 12632
Rate limit: 8 commands per 5 seconds
Client connected from 127.0.0.1:60956 (slot 0)
Client 0 received: REGISTER sithumini
Client connected from 127.0.0.1:44172 (slot 1)
Client 1 received: REGISTER alice
Client 0 received: LIST
Client 0 received: BCAST hello everyone
Client 0 received: PMSG alice hello privately
Client 0 received: JOIN general
Client 0 received: RMSG general hello room
Client 1 received: JOIN general
Client 1 received: ROOMS
Client 1 received: LEAVE general
Client 1 received: RMSG general should fail
>

centos@localhost:~/NetMessenger_IT23756632 - ./client_6632
~/NetMessenger_IT23756632
centos@localhost:~/NetMessenger_IT23756632$ ./client_6632
MSG BCAST SERVER alice joined
> LIST
>
OK USERS sithumini,alice NID:7566
> BCAST hello everyone
>
OK SENT NID:7566
> PMSG alice hello privately
>
OK SENT NID:7566
> JOIN general
>
OK JOINED general NID:7566
> RMSG general hello room
>
MSG ROOM general sithumini hello room
>
OK SENT NID:7566
> > $

centos@localhost:~/NetMessenger_IT23756632 - ./client_6632
~/NetMessenger_IT23756632
MSG PRIV sithumini hello privately
> JOIN general
>
OK JOINED general NID:7566
> > ROOMS
>
OK ROOMS general NID:7566
> > LEAVE general
>
OK LEFT general NID:7566
> RMSG general should fail
>
ERR 007 NOT_IN_ROOM NID:7566
> >

centos@localhost:~/NetMessenger_IT23756632$ cd ~/NetMessenger_IT23756632
centos@localhost:~/NetMessenger_IT23756632$ make -f Makefile_6632 clean
rm -f server_6632 client_6632
centos@localhost:~/NetMessenger_IT23756632$ make -f Makefile_6632
gcc -Wall -Wextra -pthread server_6632.c -o server_6632
gcc -Wall -Wextra -pthread client_6632.c -o client_6632
centos@localhost:~/NetMessenger_IT23756632$ ss -tlnp | grep 12632
LISTEN 0      10          0.0.0.0:12632    0.0.0.0:*      users:((("server_6632",pid=25669,fd=3))
centos@localhost:~/NetMessenger_IT23756632$
```

Figure 6.4: Broadcast, private and room messaging

The test covers BCAST, PMSG, JOIN, ROOMS, RMSG and LEAVE with two clients. alice receives the private message, both clients participate in the general room, and an RMSG after leaving is rejected with ERR 007 NOT_IN_ROOM.

```

centos@localhost:~/NetMessenger_IT23756632 - ./server_6632
rm -f server_6632 client_6632
centos@localhost:~/NetMessenger_IT23756632$ make -f Makefile_6632
gcc -Wall -Wextra -pthread server_6632.c -o server_6632
gcc -Wall -Wextra -pthread client_6632.c -o client_6632
centos@localhost:~/NetMessenger_IT23756632$ echo "NetMessenger file transfer test" > testfile.txt
centos@localhost:~/NetMessenger_IT23756632$ stat -c%u testfile.txt
43
centos@localhost:~/NetMessenger_IT23756632$ ./server_6632
NetMessenger server started.
Registration Number: IT23756632
Listening on port 12632
Client connected from 127.0.0.1:48908 (slot 0)
Client 0 received: REGISTER sithumini
Client connected from 127.0.0.1:56416 (slot 1)
Client 1 received: REGISTER alice
Client 0 received: SENDFILE alice testfile.txt 43
Stored file: ./storage/IT23756632/sithumini/testfile.txt

centos@localhost:~/NetMessenger_IT23756632 - ./client_6632
> RMSG general hello room
> MSG ROOM general sithumini hello room
> OK SENT NID:7566
> QUIT
OK BYE NID:7566
centos@localhost:~/NetMessenger_IT23756632$ nano client_6632.c
centos@localhost:~/NetMessenger_IT23756632$ ./client_6632
Connected to NetMessenger server 127.0.0.1:12632
> REGISTER sithumini
OK REGISTERED sithumini NID:7566
MSG BCAST SERVER alice joined
> SENDFILE alice testfile.txt 43
OK FILE_RECEIVED testfile.txt NID:7566

int registered;
char username[USERNAME_SIZE];
} Client;

typedef struct
{
    int active;
    char name[ROOM_NAME_SIZE];
    int members[MAX_CLIENTS];
} Room;

static... Read more
17:17

```

Figure 6.5: User-to-user file transfer and personalised storage

The server receives SENDFILE alice testfile.txt 43, stores the file below `./storage/IT23756632/sithumini/testfile.txt`, returns OK FILE_RECEIVED to the sender, and alice receives a 43-byte copy under `./received_files/`.

```

centos@localhost:~/NetMessenger_IT23756632$ cmp testfile.txt received_files/sithumini_testfile.txt
centos@localhost:~/NetMessenger_IT23756632$ echo $?
0
centos@localhost:~/NetMessenger_IT23756632$ cmp testfile.txt storage/IT23756632/sithumini/testfile.txt
centos@localhost:~/NetMessenger_IT23756632$ echo $?
0
centos@localhost:~/NetMessenger_IT23756632$

```

Figure 6.6: File integrity verification using cmp

cmp is run against both the receiving-client copy and the server-side stored copy. In both cases echo \$? returns 0, confirming byte-for-byte equality with the original testfile.txt.

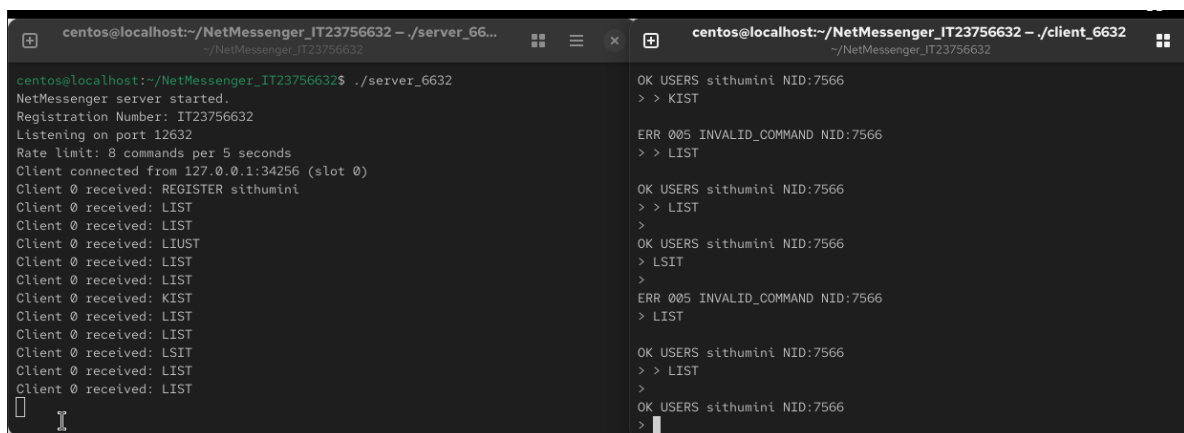
```

centos@localhost:~/NetMessenger_IT23756632$ echo $?
0
centos@localhost:~/NetMessenger_IT23756632$ cmp testfile.txt storage/IT23756632/sithumini/testfile.txt
centos@localhost:~/NetMessenger_IT23756632$ echo $?
0
centos@localhost:~/NetMessenger_IT23756632$ find storage -type f
storage/IT23756632/sithumini/testfile.txt
centos@localhost:~/NetMessenger_IT23756632$ tail -n 15 netmsg_IT23756632.log
[2026-10-07 09:45:40] Server started
[2026-10-07 09:48:14] Registered: sithumini
[2026-10-07 09:48:39] Registered: alice
[2026-10-07 09:51:49] FILE from sithumini: testfile.txt (43 bytes)
[2026-10-07 09:57:59] Graceful client disconnect
[2026-10-07 09:58:04] Graceful client disconnect
[2026-10-07 10:26:05] Server started
[2026-10-07 10:27:21] Registered: sithumini
[2026-10-07 10:29:02] Graceful disconnect: sithumini
[2026-10-07 10:37:04] Server started
[2026-10-07 10:37:35] Registered: sithumini
[2026-10-07 10:37:50] Registered: alice
[2026-10-07 10:40:23] BCAST from sithumini
centos@localhost:~/NetMessenger_IT23756632$

```

Figure 6.7: Personalised storage path and timestamped server log

The storage search shows storage/IT23756632/sithumini/testfile.txt. The log excerpt records server startup, registrations, the 43-byte file transfer, broadcast activity and graceful disconnects with timestamps.



```

centos@localhost:~/NetMessenger_IT23756632 - ./server_66...
~/NetMessenger_IT23756632
centos@localhost:~/NetMessenger_IT23756632$ ./server_6632
NetMessenger server started.
Registration Number: IT23756632
Listening on port 12632
Rate limit: 8 commands per 5 seconds
Client connected from 127.0.0.1:34256 (slot 0)
Client 0 received: REGISTER sithumini
Client 0 received: LIST
Client 0 received: LIST
Client 0 received: LIUST
Client 0 received: LIST
Client 0 received: LIST
Client 0 received: LIST
Client 0 received: KIST
Client 0 received: LIST
Client 0 received: LIST
Client 0 received: LSIT
Client 0 received: LIST
Client 0 received: LIST

centos@localhost:~/NetMessenger_IT23756632 - ./client_6632
~/NetMessenger_IT23756632
OK USERS sithumini NID:7566
> > KIST
ERR 005 INVALID_COMMAND NID:7566
> > LIST
OK USERS sithumini NID:7566
> > LIST
>
OK USERS sithumini NID:7566
> LSIT
>
ERR 005 INVALID_COMMAND NID:7566
> LIST
OK USERS sithumini NID:7566
> > LIST
>
OK USERS sithumini NID:7566
>

```

Figure 6.8: Invalid command handling

Misspelled commands such as KIST and LSIT are rejected with ERR 005

INVALID_COMMAND NID:7566. Valid LIST requests still succeed afterwards, showing that malformed input does not terminate the session.

6.2 Requirement-to-Evidence Traceability

Table 6.2: Requirement-to-evidence traceability

Requirement	Evidence
Concurrent clients	Figures 6.2-6.4
Registration and presence	Figures 6.2-6.3
Broadcast and private messaging	Figure 6.4
Chat rooms	Figure 6.4
File sharing	Figures 6.5-6.7
Personalised port / NID	Figures 6.1-6.4
Error handling	Figures 6.3, 6.4 and 6.8
Server logging	Figure 6.7
Optional extension	Figure 6.9
Incremental process evidence	Figures 8.1-8.2

7. Optional Extension: Command Rate Limiting

Rate limiting was added only after the mandatory communication features were functioning. This ordering was deliberate: the optional extension should extend a working protocol rather than conceal defects in the core implementation. The final threshold is eight text commands per five-second window for each client.

The mechanism is intentionally small and understandable. It demonstrates basic flood protection in the assignment environment without introducing a more complex token-bucket or queueing subsystem that would be disproportionate to the size of the project.

7.1 Behavioural Rationale

- Rate state is maintained per client rather than globally.
- The rate-limit response includes the personalised NID tag.
- Normal processing resumes after the time window expires.
- The extension does not disconnect the client or stop the server.

8. Incremental Development and Version Control

Git was used throughout development. The commits correspond to functional milestones rather than one final upload, which makes the progression of the implementation visible and supports the design diary.

```
Client 0 received: QUIT
^C
centos@localhost:~/NetMessenger_IT23756632$ git add server_6632.c
centos@localhost:~/NetMessenger_IT23756632$ git commit -m "Add command rate limiting and flood protection"
[main 78ce382] Add command rate limiting and flood protection
1 file changed, 859 insertions(+), 386 deletions(-)
centos@localhost:~/NetMessenger_IT23756632$ git push
Username for 'https://github.com': SithuminiGamage13
Password for 'https://SithuminiGamage13@github.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 3.60 KiB | 3.60 MiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/SithuminiGamage13/NetMessenger_IT23756632.git
d05fb24..78ce382 main -> main
centos@localhost:~/NetMessenger_IT23756632$
```

Figure 8.1: Feature commit pushed to GitHub

The screenshot records the commit and push of the rate-limiting extension to the GitHub repository, confirming that the optional feature was versioned as a distinct development step.

```
centos@localhost:~/NetMessenger_IT23756632$
centos@localhost:~/NetMessenger_IT23756632$ git log --oneline
78ce382 (HEAD -> main, origin/main) Add command rate limiting and flood protection
d05fb24 Implement file transfer framing storage and logging
5a10e09 Implement chat room management and room messaging
c2dd84d Implement broadcast and private messaging
2127c3b Add threaded multi-client support and unique usernames
454a89b Implement persistent session LIST and QUIT commands
1bcd1ae Implement client registration protocol
f7a723a Initialize personalised TCP client and server
centos@localhost:~/NetMessenger_IT23756632$
```

Figure 8.2: Incremental Git commit history

The history contains eight meaningful implementation commits covering the main stages of the project.

Table 8.1: Incremental Git development history

Commit	Development milestone
f7a723a	Initialize personalised TCP client and server
1bcd1ae	Implement client registration protocol
454a89b	Implement persistent session LIST and QUIT commands
2127c3b	Add threaded multi-client support and unique usernames
c2dd84d	Implement broadcast and private messaging
5a10e09	Implement chat room management and room messaging
d05fb24	Implement file transfer framing, storage and logging
78ce382	Add command rate limiting and flood protection

8.1 Development Corrections

The development process also included correction of compiler feedback. An unused helper and an output-length warning were addressed before the final build evidence was captured. This is recorded in Appendix B rather than being hidden from the development history.

9. Design Decisions, Assumptions and Limitations

9.1 Design Decisions

A thread-per-client model was chosen because it maps directly to the simultaneous-client requirement and keeps each connection's command loop understandable. An event-driven design could scale to more connections, but it would add complexity without improving the learning objectives of this assignment.

Text commands and file bytes use separate receive logic. This avoids the common TCP error of treating `recv()` boundaries as message boundaries.

9.2 Assumptions

- Testing was performed on localhost, so host firewall and external network routing were outside the functional test.
- Usernames, room names and command lines remain within the fixed buffer limits used by the program.
- A practical maximum file-size check is used to avoid unbounded transfers in the assignment environment.
- The receiving client uses an implementation-specific file header before raw bytes so it knows the sender, filename and length.

9.3 Limitations

- The application is an academic prototype and does not provide encryption or identity authentication beyond the chosen username.
- Client and room tables are fixed-size rather than dynamically allocated.
- Logs and transferred files are stored locally rather than in a database.
- The rate limiter is basic and should not be considered production-grade denial-of-service protection.

10. Verification Quality and Learning

The strongest part of the final verification was using independent checks instead of relying only on successful-looking command output. Port 12632 was verified with `ss`, file equality was checked with `cmp`, the personalised storage path was inspected with `find`, and the server log was read directly. These checks make the evidence stronger than screenshots of commands alone.

Negative cases were also tested deliberately. Duplicate registration, malformed commands, invalid room use and room messaging after LEAVE were exercised to confirm that the server rejects bad input without terminating unrelated sessions.

The main technical learning from the implementation was that TCP programming is not only about socket creation and `send/recv` calls. Correct behaviour also depends on framing, exact byte counts, shared-state protection, clean disconnection handling and testing the behaviour of several terminals at the same time.

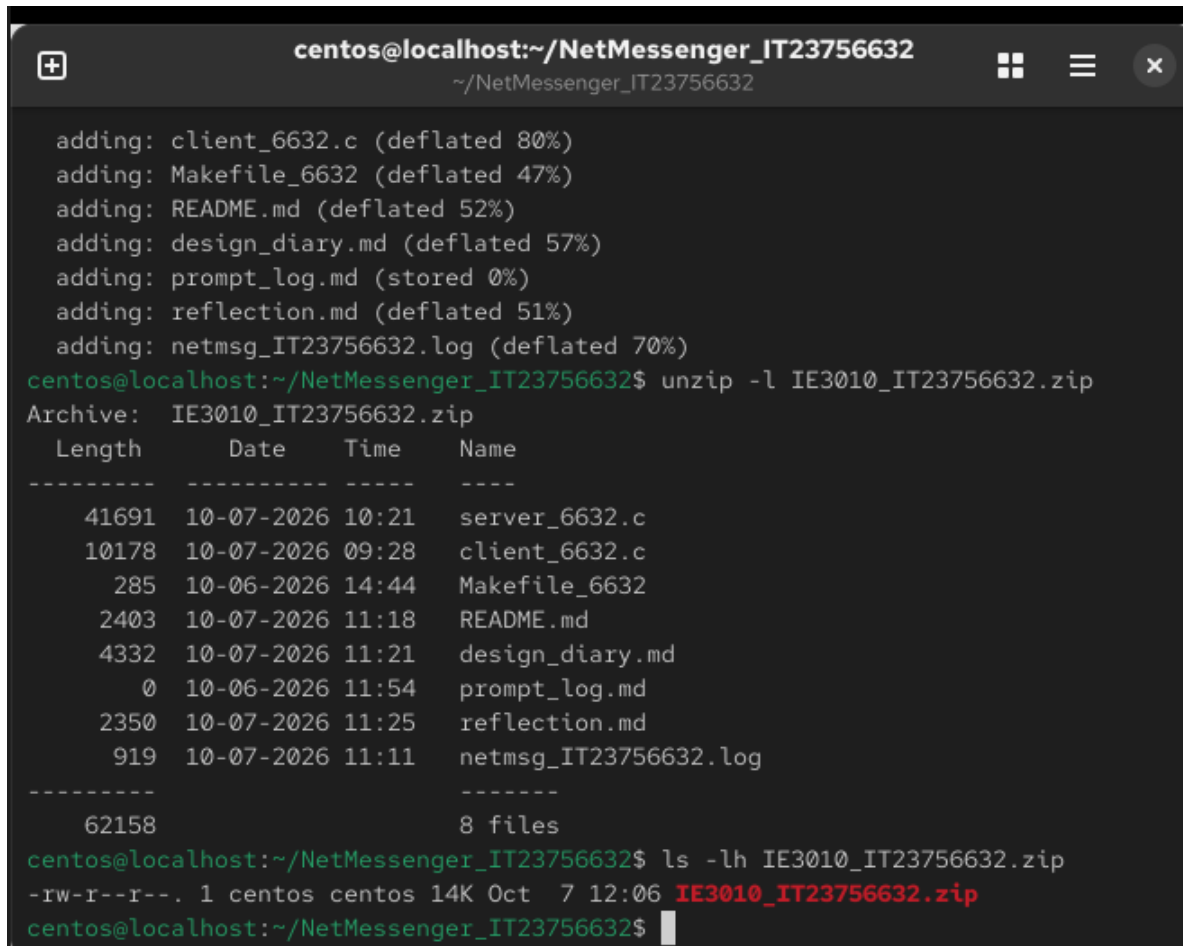
11. Conclusion

The final NetMessenger implementation meets the intended scope of the assignment: a personalised TCP server, persistent multi-client sessions, unique registration, connected-user listing, broadcast and private messaging, room operations, file transfer, protocol error handling, timestamped logging and disconnect cleanup. The optional per-client rate limiter extends the server without replacing the mandatory functionality.

The evidence in this report is drawn from the actual CentOS development environment and demonstrates both normal operation and controlled handling of invalid cases. The project provided practical experience with TCP framing, exact-length data transfer, concurrency, mutex-protected shared state and evidence-based debugging.

Appendix A - Submission Package Evidence

The final archive is named IE3010_IT23756632.zip. The screenshot below records the archive-content check performed with `unzip -l`. At the time this screenshot was captured, the report was still being revised, so the archive contains the eight project/support files and the report remains a separate deliverable until finalisation.

A terminal window titled 'centos@localhost:~/NetMessenger_IT23756632' with a subtitle '~/.NetMessenger_IT23756632'. The terminal shows the process of adding files to a ZIP archive and then listing its contents. The files added are: client_6632.c (deflated 80%), Makefile_6632 (deflated 47%), README.md (deflated 52%), design_diary.md (deflated 57%), prompt_log.md (stored 0%), reflection.md (deflated 51%), and netmsg_IT23756632.log (deflated 70%). The command 'unzip -l IE3010_IT23756632.zip' is executed, resulting in a table of archive contents. The table has columns for Length, Date, Time, and Name. The files listed are: server_6632.c (41691 bytes, 10-07-2026 10:21), client_6632.c (10178 bytes, 10-07-2026 09:28), Makefile_6632 (285 bytes, 10-06-2026 14:44), README.md (2403 bytes, 10-07-2026 11:18), design_diary.md (4332 bytes, 10-07-2026 11:21), prompt_log.md (0 bytes, 10-06-2026 11:54), reflection.md (2350 bytes, 10-07-2026 11:25), and netmsg_IT23756632.log (919 bytes, 10-07-2026 11:11). The total size is 62158 bytes for 8 files. The command 'ls -lh IE3010_IT23756632.zip' is also executed, showing the file as 14K, dated Oct 7 12:06, with permissions -rw-r--r--.

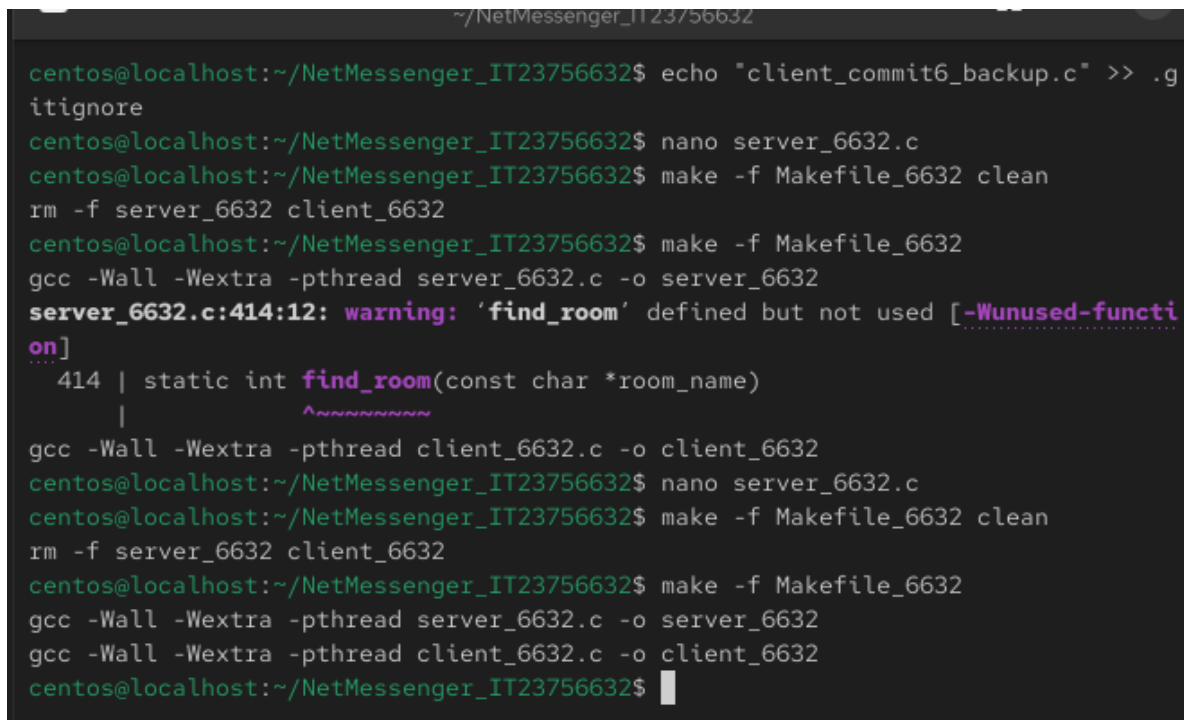
```
centos@localhost:~/NetMessenger_IT23756632
~/NetMessenger_IT23756632
adding: client_6632.c (deflated 80%)
adding: Makefile_6632 (deflated 47%)
adding: README.md (deflated 52%)
adding: design_diary.md (deflated 57%)
adding: prompt_log.md (stored 0%)
adding: reflection.md (deflated 51%)
adding: netmsg_IT23756632.log (deflated 70%)
centos@localhost:~/NetMessenger_IT23756632$ unzip -l IE3010_IT23756632.zip
Archive:  IE3010_IT23756632.zip
  Length      Date    Time    Name
-----
  41691  10-07-2026  10:21   server_6632.c
  10178  10-07-2026  09:28   client_6632.c
    285  10-06-2026  14:44   Makefile_6632
   2403  10-07-2026  11:18   README.md
   4332  10-07-2026  11:21   design_diary.md
     0   10-06-2026  11:54   prompt_log.md
   2350  10-07-2026  11:25   reflection.md
    919  10-07-2026  11:11   netmsg_IT23756632.log
-----
  62158                      8 files
centos@localhost:~/NetMessenger_IT23756632$ ls -lh IE3010_IT23756632.zip
-rw-r--r--. 1 centos centos 14K Oct  7 12:06 IE3010_IT23756632.zip
centos@localhost:~/NetMessenger_IT23756632$
```

Figure A.1: Final submission ZIP content check

The archive contains `server_6632.c`, `client_6632.c`, `Makefile_6632`, `README.md`, `design_diary.md`, `prompt_log.md`, `reflection.md` and `netmsg_IT23756632.log`. The final report can be added to the archive after the final document is approved.

Appendix B - Supporting Development Evidence

The following screenshot records a compiler warning that was identified and removed during development. It is included to show the correction process rather than presenting the final clean build as if no debugging was required.



```
~/NetMessenger_IT23756632
centos@localhost:~/NetMessenger_IT23756632$ echo "client_commit6_backup.c" >> .gitignore
centos@localhost:~/NetMessenger_IT23756632$ nano server_6632.c
centos@localhost:~/NetMessenger_IT23756632$ make -f Makefile_6632 clean
rm -f server_6632 client_6632
centos@localhost:~/NetMessenger_IT23756632$ make -f Makefile_6632
gcc -Wall -Wextra -pthread server_6632.c -o server_6632
server_6632.c:414:12: warning: 'find_room' defined but not used [-Wunused-function]
...
414 | static int find_room(const char *room_name)
    |             ^~~~~~
gcc -Wall -Wextra -pthread client_6632.c -o client_6632
centos@localhost:~/NetMessenger_IT23756632$ nano server_6632.c
centos@localhost:~/NetMessenger_IT23756632$ make -f Makefile_6632 clean
rm -f server_6632 client_6632
centos@localhost:~/NetMessenger_IT23756632$ make -f Makefile_6632
gcc -Wall -Wextra -pthread server_6632.c -o server_6632
gcc -Wall -Wextra -pthread client_6632.c -o client_6632
centos@localhost:~/NetMessenger_IT23756632$
```

Figure B.1: Compiler warning identified and corrected before the final build
The first compilation reports an unused find_room helper. The source is then edited and rebuilt. The final clean-build result used as marking evidence is shown in Figure 6.1.

Appendix C - Annotated Source-Code Evidence

The assignment requires legible evidence of the key source-code areas. The following extracts are reproduced from the final NetMessenger implementation so that the important logic can be reviewed without reducing the code to unreadable full-page listings.

```
1  /* Accept clients and create one handler thread per connection. */
2  while (1) {
3      int client_fd = accept(server_fd,
4                          (struct sockaddr *)&client_addr, &client_len);
5
6      if (client_fd < 0) {
7          perror("accept");
8          continue;
9      }
10
11     int index = add_client(client_fd);
12     if (index < 0) {
13         const char *full_message =
14             "ERR 006 SERVER_FULL " NID_TAG "\n";
15         send_all(client_fd, full_message, strlen(full_message));
16         close(client_fd);
17         continue;
18     }
19
20     pthread_t thread;
21     int *thread_index = malloc(sizeof(int));
22     *thread_index = index;
23
24     if (pthread_create(&thread, NULL,
25                     handle_client, thread_index) != 0) {
26         perror("pthread_create");
27         free(thread_index);
28         close(client_fd);
29         remove_client(index);
30         continue;
31     }
32
33     pthread_detach(thread);
34 }
```

Figure C.1: Server accept loop and pthread-based client concurrency

The main server loop accepts a TCP connection, allocates a client slot and creates a detached pthread that runs `handle_client()`. This is the point where the thread-per-client concurrency model is established.

```

1  /* Read one complete newline-terminated text command. */
2  ssize_t read_line(int sockfd, char *buffer, size_t max_length)
3  {
4      size_t index = 0;
5
6      while (index < max_length - 1) {
7          char ch;
8          ssize_t received = recv(sockfd, &ch, 1, 0);
9
10         if (received == 0) return 0;
11         if (received < 0) return -1;
12         if (ch == '\n') break;
13
14         if (ch != '\r')
15             buffer[index++] = ch;
16     }
17
18     buffer[index] = '\0';
19     return (ssize_t)index;
20 }
21
22 /* Receive exactly the required number of raw file bytes. */
23 int receive_exact(int sockfd, unsigned char *buffer, size_t length)
24 {
25     size_t total_received = 0;
26
27     while (total_received < length) {
28         ssize_t received = recv(sockfd,
29             buffer + total_received,
30             length - total_received, 0);
31
32         if (received <= 0) return -1;
33         total_received += (size_t)received;
34     }
35
36     return 0;
37 }

```

Figure C.2: Line framing and exact-byte file reception

`read_line()` reads one newline-terminated command without assuming `recv()` returns a complete line. `receive_exact()` continues reading until the requested file byte count has been received, which is essential for correct SENDFILE framing.

```

1  /* BCAST */
2  else if (strncmp(buffer, "BCAST ", 6) == 0) {
3      char *message = buffer + 6;
4
5      if (strlen(message) == 0) {
6          snprintf(response, sizeof(response),
7              "ERR 005 INVALID_FORMAT %s\n", NID_TAG);
8      } else {
9          broadcast_message(index, message);
10         snprintf(response, sizeof(response),
11             "OK SENT %s\n", NID_TAG);
12     }
13
14     send_all(client_fd, response, strlen(response));
15 }
16
17 /* PMSG */
18 else if (strncmp(buffer, "PMSG ", 5) == 0) {
19     char *content = buffer + 5;
20     char *space = strchr(content, ' ');
21
22     if (space == NULL) {
23         snprintf(response, sizeof(response),
24             "ERR 005 INVALID_FORMAT %s\n", NID_TAG);
25     } else {
26         *space = '\0';
27         char *target_username = content;
28         char *message = space + 1;
29
30         if (!private_message(index, target_username, message))
31             snprintf(response, sizeof(response),
32                 "ERR 002 USER_NOT_FOUND %s\n", NID_TAG);
33         else
34             snprintf(response, sizeof(response),
35                 "OK SENT %s\n", NID_TAG);
36     }
37
38     send_all(client_fd, response, strlen(response));
39 }

```

Figure C.3: Broadcast and private-message command handling
The command dispatcher validates BCAST and PMSG input, calls the relevant delivery routine and returns the appropriate NID-tagged OK or ERR response to the sender.

```

1 int room_message(int sender_index,
2                 const char *room_name,
3                 const char *message)
4 {
5     int target_sockets[MAX_CLIENTS];
6     int target_count = 0;
7
8     pthread_mutex_lock(&rooms_mutex);
9     int room_index = find_room_locked(room_name);
10
11     if (room_index < 0) {
12         pthread_mutex_unlock(&rooms_mutex);
13         return -1;
14     }
15
16     if (!rooms[room_index].members[sender_index]) {
17         pthread_mutex_unlock(&rooms_mutex);
18         return 0;
19     }
20
21     pthread_mutex_lock(&clients_mutex);
22
23     for (int i = 0; i < MAX_CLIENTS; i++) {
24         if (i != sender_index &&
25             rooms[room_index].members[i] &&
26             clients[i].registered) {
27             target_sockets[target_count++] =
28                 clients[i].socket_fd;
29         }
30     }
31
32     pthread_mutex_unlock(&clients_mutex);
33     pthread_mutex_unlock(&rooms_mutex);
34 }

```

Figure C.4: Room membership check and room-message routing
`room_message()` resolves the room, verifies that the sender is a member, gathers the sockets of registered room members and forwards the room message only to those targets. Shared room and client state is protected by mutexes.

```

1  if (strcmp(buffer, "SENDFILE ", 9) == 0) {
2      char target[MAX_USERNAME], room_name,
3      char filename[MAX_FILENAME], message;
4      {
5          unsigned long long file_size_value;
6          char *clients[30][MAX_CLIENTS];
7          int target_count = 0;
8          int fields = sscanf(buffer,
9          pthread_mutex_lock(&rooms_mutex),
10         int target_index, len, name, &room_name);
11
12         if (fields < 3) {
13             pthread_mutex_unlock(&rooms_mutex);
14             return ERR105 INVALID_FORMAT "%s\n", NID_TAG);
15         }
16         send_all(client_fd, response, strlen(response));
17         continue;
18     }
19     if (!rooms[room_index].members[sender_index]) {
20         pthread_mutex_unlock(&rooms_mutex);
21         if (file_size_value >
22             (unsigned long long)MAX_FILE_SIZE) {
23             pthread_mutex_lock(&clients_mutex);
24             for (int i = 0; i < MAX_CLIENTS; i++) {
25                 if (i != sender_index &&
26                     rooms[room_index].members[i] &&
27                     !strcmp(response,
28                         "ERR004 EXCEEDED TARGET COUNT\n", NID_TAG);
29                     clients[i].socket_fd;
30                 send_all(client_fd, response, strlen(response));
31             }
32             continue;
33         }
34         pthread_mutex_unlock(&clients_mutex);
35         pthread_mutex_lock(&rooms_mutex);
36         file_size_value;
37     }
38 }

```

Figure C.5: SENDFILE parsing, size validation and framing protection
The SENDFILE branch parses the target, filename and declared byte count. Oversized or malformed transfers are rejected, and rejected file bytes are consumed so the TCP stream remains aligned for the next command.

```

1  /* Invalid command handling */
2  else {
3      snprintf(response, sizeof(response),
4          "ERR 005 INVALID_COMMAND %s\n", NID_TAG);
5
6      send_all(client_fd, response, strlen(response));
7
8      write_log(
9          "INVALID_COMMAND username=%s command=\"%s\"",
10         registered_username, buffer);
11 }
12
13 /* Common cleanup after QUIT or socket closure */
14 close(client_fd);
15 remove_client(index);
16 return NULL;

```

Figure C.6: Invalid-command response and common connection cleanup
Unknown commands generate ERR 005 INVALID_COMMAND with the personalised NID tag and are logged. After QUIT or socket termination, the client socket is closed and remove_client() clears the associated server state.